

[Challenge 4] Smart Stadiums & Tournament Operations

Build a GenAI-enabled solution that enhances stadium operations and the overall tournament experience for fans, organizers, volunteers, or venue staff. The solution must leverage Generative AI to improve navigation, crowd management, accessibility, transportation, sustainability, multilingual assistance, operational intelligence, or real-time decision support during the FIFA World Cup 2026.

Program Overview

What is Prompt Wars? A bi-weekly challenge for individual developers to build applications using prompt-driven development (GenAI).

Submission Requirements: A working deployment link and a LinkedIn post explaining the development strategy.

Evaluation: Submissions are judged on code quality, security, efficiency, testing, accessibility, and problem statement alignment. Participants get three attempts, with only the latest one counting.

Challenge 4: Smart Stadiums & Tournament Operations

The Goal: Use GenAI to solve real-world problems for a specific persona in a stadium environment (e.g., FIFA World Cup context).

Key Personas: You should choose one specific persona to focus on: Fans, Organizers, Volunteers, or Venue Staff.

Development Focus: Pick one or two "verticals"—such as navigation, crowd management, accessibility, or multilingual assistance—rather than trying to solve everything.

Engineering Tip: The jury evaluates whether GenAI is used for reasoning and decision support, not just as a basic chatbot.

Rewards & Credits

Prompt Credits (PC): Earned through bi-weekly submissions and leaderboard rankings. Credits accumulate over the program year and can be redeemed in the swag store.

Important Reminders: Credits expire on January 1st, 2027. Only the top 400 participants per challenge earn leaderboard credits.

Q&A Highlights

Data Sources: Use public APIs or generate synthetic data for demos. The application must include an interface for the jury to test it with their own data.

Code Quality: Focus on optimized code and handling edge cases rather than just total lines of code.

Multilingual: The AI should understand the user's context (e.g., current location or travel plans) to provide relevant language and assistance.

The main goal of the LinkedIn post is to document your development approach for the Prompt Wars Virtual challenge. This documentation is mandatory and serves several purposes:

Showcase your learning: It ensures participants are adopting GenAI tools for genuine learning rather than just surface-level usage.

Explain your strategy: You must detail the tools you chose and why, how your prompts evolved throughout your submission attempts, and specifically what parts of the project were handled by GenAI versus what you designed yourself.

In the context of the Prompt Wars Virtual challenge, "reasoning" is a critical differentiator for high-quality Generative AI projects. Here is why it is essential:

Moving Beyond Basic Chatbots: The video explains that simply building a chatbot that answers surface-level questions is insufficient. Effective AI projects use models to reason over data, rather than just retrieving or outputting static information.

Explainability (XAI): Reasoning is what enables Explainable AI. It allows the application not just to provide an output (like a recommendation or an instruction), but to provide the logic or reasoning behind why that specific output was generated.

Actionable Decision Support: For complex environments like smart stadiums, reasoning allows the AI to analyze raw data (such as crowd density or safety thresholds) and translate it into plain English recommendations for operators, which is far more valuable than presenting raw numbers.

Contextual Adaptability: Reasoning helps an AI understand the intent behind a user request. For example, knowing the difference between a user asking for a washroom for general convenience versus someone in medical distress requires the AI to process context and determine the appropriate response tone and urgency.

Essentially, the jury evaluates projects based on whether the AI performs a meaningful task that would be difficult or impossible with standard rule-based code, focusing on the ability to connect input, reasoning, and actionable outcomes.

The cumulative leaderboard is designed to foster long-term engagement and consistent participation throughout the year, rather than focusing solely on isolated, one-off project submissions.

Here is how it encourages ongoing involvement:

Consistency Matters: Because the leaderboard aggregates points across all bi-weekly challenges, participants are motivated to show up regularly. A single strong project is not enough to dominate; long-term commitment is required to climb and maintain a high rank.

Compounding Rewards: Prompt Credits (PC) are earned with every valid submission and top-400 ranking. These credits stack over time, allowing even participants who aren't always in the top three to accumulate enough credits to redeem for significant rewards from the swag store.

Year-long Motivation: The program is structured as a year-long initiative, ensuring that participants remain active and motivated across multiple challenge cycles. Since credits do not expire within the program year (until January 1st, 2027), users have the flexibility to save up for higher-tier prizes, which encourages continued participation in future challenges.

For the Prompt Wars Virtual Challenge 4, the rules regarding submission attempts are as follows:

Attempt Limit: Each participant is allowed a maximum of three submission attempts.

Latest Attempt Counts: Only your latest submission attempt is counted toward the final leaderboard ranking.

Example: If you score 90 on your first attempt, 95 on your second, but then receive a 60 on your third attempt, the score of 60 will be the one used for the leaderboard.

Therefore, you are encouraged to use your attempts wisely.

For the Prompt Wars Virtual challenge, effective prompt engineering is key to success. According to the session, here are the best practices for formatting your prompts:

Structured Prompting: Follow a logical structure that includes system instructions, user instructions, few-shot/many-shot examples (providing examples of desired input/output), and a clearly defined output format.

Use JSON Format: The speaker highly recommends providing your prompts in JSON format. This helps the model maintain a consistent structure and improves the reliability of the output.

Learning Resources: If you are new to this process, the speaker suggests spending 20–30 minutes learning the fundamentals of prompt engineering. You can find additional resources and workshops on the Hack2skill dashboard under the Interactions tab.

The explainer session does not specify any browser requirements for deploying or viewing your application.

Instead, the focus is on the deployment platform and accessibility. The speaker notes that you are free to use various hosting services—such as Vercel, Netlify, or Google Cloud Run—to make your project accessible via a deployment link.

Crucially, because the jury will manually evaluate your submission, it is important that your project is fully functional and accessible through a standard web link, avoiding any static or hard-coded pages that might fail to render or interact correctly during the assessment process.

To succeed in Challenge 4: Smart Stadiums and Tournament Operations and secure a top 400 ranking, focus on the following core requirements and strategic priorities:

1. Master the Core Workflow (Persona + Problem)

Pick one Persona: Do not try to solve for everyone. Focus deeply on either fans, organizers, volunteers, or venue staff.

Select specific verticals: Don't attempt all eight problem areas. Pick one or two (e.g., navigation, crowd management, or multilingual assistance) and build a focused solution.

Solve a real problem: The jury looks for projects that improve a specific, chaotic moment (like bottlenecks or language barriers) rather than a generic app.

2. Implement "Reasoning" (Not just Chatbots)

Move beyond surface-level: The AI must reason over data (e.g., analyzing crowd density and suggesting a path) rather than just providing static information.

Explainable AI (XAI): Your application should be able to justify why it made a specific recommendation or decision.

3. Technical Requirements for Submission

GenAI is mandatory: Ensure GenAI is deeply integrated into your project. Static or hard-coded pages will result in disqualification.

Test your inputs: The jury will manually test your app. Ensure you have a mechanism (like a file upload feature for CSV/PDF) where the jury can input their own data to verify your solution works dynamically.

Deployment: You must provide a working, live deployment link. You are free to use platforms like Vercel, Netlify, or Google Cloud Run.

4. Strategic Rules for Scoring

Submission limit: You get exactly three attempts. Only your latest attempt counts toward your leaderboard score.

Documentation: Your LinkedIn post is just as important as the code. Explain your strategy, how your prompts evolved, and what specific challenges you overcame.

Optimize your code: Focus on efficiency and best practices. Avoid unnecessary complexity; clean, well-tested code that handles edge cases will score higher in automated evaluations.

5. Final Checklist

Prompt Engineering: Use structured prompts (system/user instructions, examples, and JSON output format) to ensure reliability.

Consistency: Credits are cumulative. Even if you don't hit the top 3, consistent participation in every challenge keeps you on the leaderboard.

Missing the first three rounds puts you at a points deficit, but you can absolutely still catch up and secure those Google developer swags. The leaderboard rewards consistency, high quality, and community engagement over just a one-time win.

To maximize your points and skyrocket your rank starting with **Challenge #4**, focus heavily on the official grading criteria:

1. Maximize Your Project Delivery Score (70% of Base Points)

- **Build Beyond the Prompt:** Do not just build a bare-minimum app. Add a cohesive User Interface (UI) using clean CSS or a framework like Tailwind.

- **Flawless Functionality:** Ensure your app has zero broken buttons or console errors. The judging panel tests the live deployed link thoroughly.
- **Edge Case Handling:** Add simple validation (e.g., loading states, error messages if an API fails) to make your app feel like a premium, production-ready product.

2. Gamify the "Build-In-Public" Rule (30% of Base Points)

Your LinkedIn submission is just as important as your code. The organizers reward posts that generate professional traction.

- **Use Video, Not Just Screenshots:** Record a quick **30 to 60-second screen recording** of your app working. Posts with videos get significantly higher algorithmic reach and higher scores from reviewers.
- **Document the "Vibe Coding" Journey:** Explain exactly *how* you used the AI. Detail the specific prompts you used, what failed, and how you iterated to fix it.
- **Tag Correctly:** Always include the mandatory event tags (like [#PromptWars2026](#), [#Hack2skill](#), and [#GoogleForDevelopers](#)) so the tracking bots log your post.

3. Claim Every Possible Bonus Point

- **Submit Early:** Don't wait for the final day. Early submissions often receive a speed bonus and get noticed first by the community.
- **Engage in the Discord/Community:** Help other developers troubleshoot their setups in the [PromptWars Community Hub](#). Active community members frequently get highlighted, driving more organic traffic to their LinkedIn posts.
- **Maintain a Streak:** The leaderboard penalizes dropping out. Commit to submitting *every single challenge* from #4 until the final drop in November. Consistent average scores will easily beat out players who get a high score once and stop.

What is the **specific problem statement or theme** for Challenge #4? Tell me what you need to build, and we can **architect the app** and **write the foundational prompts** right now to get you ahead of the competition!

The timeline is tight, but **5 days is more than enough time** to build a high-scoring

app using "vibe coding," since you are leveraging AI for the heavy lifting rather than coding everything by hand.

Since **Challenge #4** focuses on **Smart Stadiums & Tournament Operations for the FIFA World Cup 2026**, you need to build a Genie-powered agentic application. It should optimize complex operations like multi-language attendee assistance, real-time crowds/decision support, or stadium resource allocation. [1, 2]

The blueprint below details exactly how to structure your 5-day sprint to secure a podium finish:

Day 1: Define Your Feature Scope & Set Up the Stack

Do not try to build an entire operating system. Focus on 2–3 highly polished, high-utility features that judges will love.

- **Core Idea:** A **Smart Tournament Command Center** featuring a real-time event coordinator chat agent (handling venue changes or language translation), a dynamic crowd traffic visualizer, and an incident reporting dashboard. [2]
- **The Stack:** Use **Google Antigravity** [1.51] or Google AI Studio for your backend LLM orchestration, and prompt an AI generator like v0.dev or Claude Engineer to spin up a clean **Next.js/React frontend with Tailwind CSS**.

Day 2: Build the Core AI Agent Architecture

Spend today locking down the technical agent intelligence, which makes up the bulk of your score.

- **Prompt Orchestration:** Structure your prompts using **System Instructions** inside Google AI Studio/Antigravity to give your agent a distinct role (e.g., *"You are an elite FIFA World Cup Stadium Operations AI agent..."*).
- **Multi-Language Capabilities:** Build explicit multi-language translation safety overrides into your prompt system so an attendee can ask questions in Spanish, Arabic, or Hindi and get instant, location-accurate stadium directions. [2]

Day 3: Frontend Integration & Deployment

Connect your AI agent backend to your user interface and get the app live early to avoid deadline-day server panic.

- **UI Polish:** Make sure the UI has clear, stadium-themed branding (football graphics, clean tables, loading indicators).
- **Live Deployment:** Deploy the frontend immediately to a public platform like **Vercel**, **Netlify**, or **Google Cloud Run**. Make sure the URL works perfectly on both desktop and mobile devices. [2]

Day 4: Edge Case Polish & Video Recording

The difference between an average submission and a top-tier swag winner comes down to the final details and your presentation.

- **Break It On Purpose:** Test what happens when a user inputs nonsense data into your stadium coordinator tool. Add error-handling prompts so the AI gently redirects them.
- **The Pitch Video:** Do not just submit a silent screen share. Record a **45-second walkthrough video** with your voice explaining the precise problem your app solves, demonstrating a live query, and showing how the AI processed it.

Day 5: The Build-in-Public LinkedIn Launch

Spend your final day crafting the narrative that handles 30% of your total base score. [3]

- **Write the Post:** Use a structured format on LinkedIn detailing the challenge name, your architecture layout, what prompt constraints worked best, and what you learned while vibe-coding.
- **Final Check:** Ensure your **GitHub repository is public**, your live link is completely unfettered, and you have tagged [#PromptWars2026](#), [#Hack2skill](#), and [#GoogleForDevelopers](#). Submit the form on the [PromptWars Portal](#) well before the July 19th cutoff. [2, 4]

Would you like to start right now by **writing the system instructions prompt for your FIFA World Cup Assistant Agent**, or would you prefer a **template for your Day 5 LinkedIn post** so you can plan backward?

[1] <https://www.youtube.com>

[2] <https://www.youtube.com>

[3] <https://promptwars.in>

[4] <https://www.instagram.com>